Implement Client Script and UI Policy Incident

Code Layout Readability and Reusability

Prepared for the ServiceNow Incident client-side controls micro project

ServiceNow Incident micro project that uses a UI Policy and Client Scripts to enforce high-impact Incident controls, improve data quality, and prevent invalid submissions.

Project record

Template area	Completed project content
Context and need	Use descriptive names: High Impact Control, Auto set urgency for high impact, and Prevent save if Assigned To missing.
Selected approach	Use early returns for isLoading and empty values in the onChange script.
Implementation detail	Keep conditions explicit: impact equals 1 and assigned_to is empty.
Expected outcome	Use g_form APIs consistently: setValue, addInfoMessage, getValue, and showErrorBox.

Practical interpretation

The controls are intentionally conditional. When an agent marks an Incident as High impact, the form helps them complete the record correctly instead of relying on memory alone. When the impact is changed back to Medium or Low, the UI Policy reverses the extra restrictions so that the normal Incident workflow remains usable.

This balance is important in a service desk setting: urgent records need stronger guidance, while routine records should not be burdened with unnecessary fields or messages.

Implement Client Script and UI Policy Incident

Code Layout Readability and Reusability - Implementation Notes

How this template applies to the Incident configuration

Work item	Project-specific evidence	Acceptance check
Context and need	Configuration is documented in the ServiceNow Incident table and related System UI records.	Record is configured and active.
Selected approach	The rule is applied only when the form condition is true, avoiding an always-on restriction.	Condition responds to Impact = High.
Implementation detail	The expected browser behavior can be observed directly during a new Incident test.	Agent receives the expected field behavior or message.
Expected outcome	The result supports more reliable triage, routing, and SLA reporting.	A valid record can be saved after the required action.

Validation notes

Testing should begin with a new Incident. Set Impact to High, leave Assigned To empty, and submit the form. The save must be blocked and the user must see a clear error. Next, populate Assigned To and submit again to confirm the record saves. Finally, change Impact away from High and verify that the reversible UI Policy behavior has been removed.

For list editing, double-click State from the Incident list. The onCellEdit script should show its warning and reject the edit. Opening the same Incident form and updating State there should still be allowed. These tests demonstrate that the design protects data quality without preventing legitimate work.